HTML & CSS

Da estrutura à experiência.

Como transformar conteúdo em páginas claras, bonitas e responsivas.

ESTRUTURA | ESTILO | RESPONSIVIDADE

index.html + style.css

primeiro contato

```
01  <h1>Olá, web!</h1>
02
03  h1 { color: #6ce5d2; }
```

Olá, web!

O navegador conecta as duas camadas.

HTML é uma linguagem de marcação. CSS é uma linguagem de estilos.

</>

HTML

Conteúdo e significado: títulos, links e formulários.

+

{ }

CSS

Apresentação e layout: cores, espaços e posições.

→

↗

PÁGINA

O navegador interpreta os arquivos e renderiza a interface.

JavaScript pode adicionar comportamento; nesta apresentação, o foco é HTML e CSS.

Onde isso aparece no mundo real?

Os mesmos fundamentos sustentam páginas simples e interfaces de sistemas complexos.

01

Sites e portfólios

Conteúdo organizado, identidade visual e navegação clara.

PRESENÇA DIGITAL

02

Sistemas e painéis

Formulários, menus, tabelas e interfaces administrativas.

USO PROFISSIONAL

03

Lojas e serviços

Catálogo em cards, busca de produtos e telas de cadastro.

EXPERIÊNCIA DO USUÁRIO

HTML: o esqueleto de uma página.

exemplo.html

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>Minha página</title>
</head>
<body>
  <h1>Olá, web!</h1>
</body>
</html>
```

01 Documento

doctype declara HTML moderno; lang indica o idioma.

02 Metadados

head reúne título, codificação e configuração da viewport.

03 Conteúdo visível

body contém os elementos apresentados na página.

Tags: escolha pelo significado.

Semântica descreve a função do conteúdo e ajuda pessoas, ferramentas e mecanismos de busca.



h1 … h6 / p

Títulos em hierarquia e parágrafos.

a / img

Links com href; imagens com src e alt.

ul / ol / li

Listas sem ordem ou numeradas.

div / span

Agrupamentos sem significado específico.

Use tags semânticas quando houver uma função clara.

Formulários: transforme campos em dados.

exemplo.html

```
<form action="/cadastro" method="post">
  <label for="email">E-mail</label>
  <input id="email" name="email"
         type="email" required>

  <label for="curso">Curso</label>
  <select id="curso" name="curso">
    <option>HTML e CSS</option>
  </select>

  <button type="submit">Enviar</button>
</form>
```

01 **Identificação**

for conecta o rótulo ao id. name identifica o dado enviado.

02 **Validação nativa**

type="email" e required ajudam o navegador a validar.

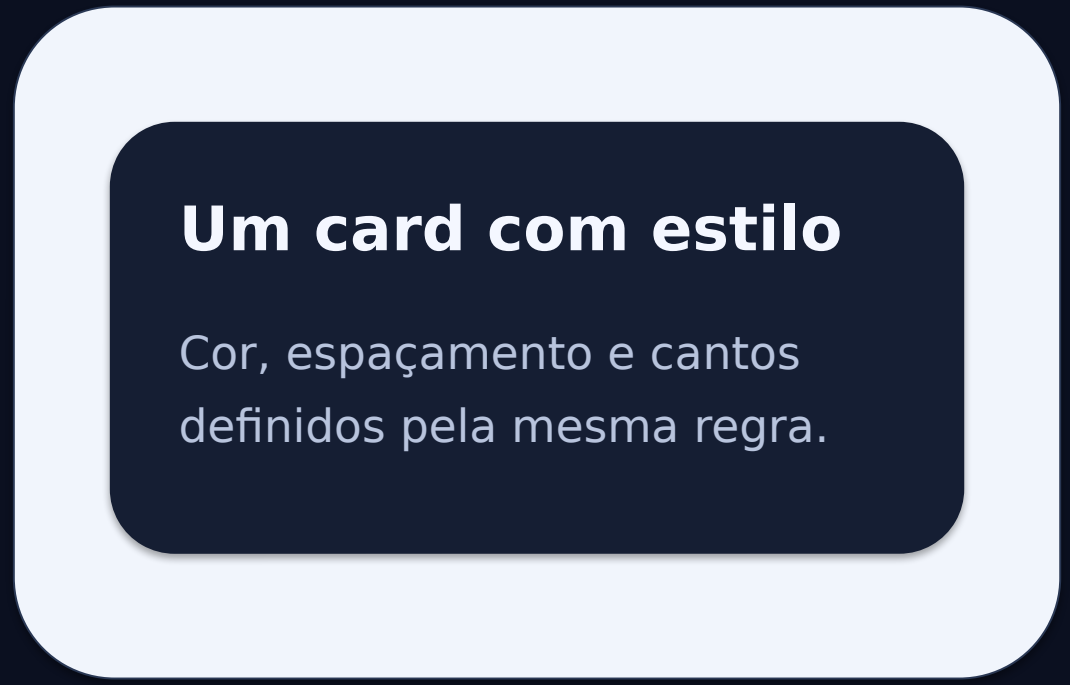
03 **Envio e servidor**

action aponta o destino. O servidor deve validar e salvar.

CSS: selecione, declare e transforme.

style.css

```css
.card {
  background: #151e33;
  color: #f4f7ff;
  padding: 24px;
  border-radius: 18px;
}
```



No HTML: `<link rel="stylesheet" href="style.css">`

Seletores: quem recebe o estilo?

Classes podem se repetir. O valor de um id deve ser único no documento.

SELETOR	EXEMPLO	O QUE SELECIONA
Tipo	`p { … }`	Todos os parágrafos.
Classe	`.card { … }`	Elementos com class="card".
ID	`#inscricao { … }`	O elemento com id="inscricao".
Atributo	`input[type="email"]`	Campos com esse atributo e valor.
Pseudoclasse	`button:focus-visible`	Botão em estado de foco visível.

```
Exemplo: <article class="card destaque" id="curso-html">…</article>
```

Quando duas regras disputam o mesmo estilo.

style.css

```
01 p { color: white; }
02
03 .aviso { color: #6ce5d2; }
04
05 #mensagem { color: #ffaa88; }
```

index.html

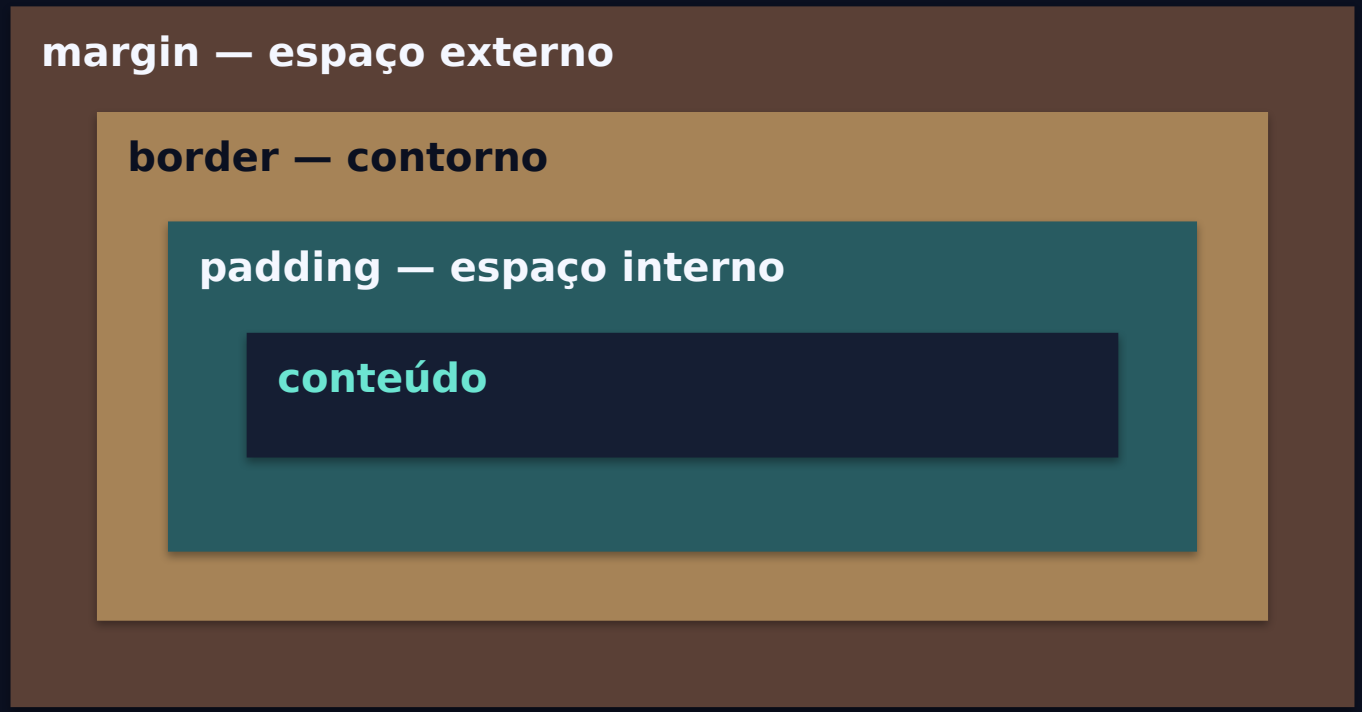
```
01 <p id="mensagem" class="aviso">Olá!</p>
```

Olá!

Neste exemplo, o seletor de ID tem maior especificidade.

No mesmo contexto de cascata: ID > classe > tipo. Em empate, vence a última regra.

Box model: cada elemento ocupa uma caixa.



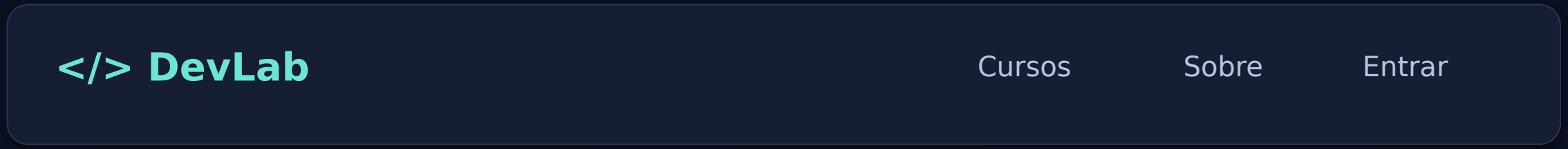
style.css

```
01  * {
02    box-sizing: border-box;
03  }
```

A largura declarada inclui conteúdo, padding e borda.

Flexbox: organize uma direção de cada vez.

Ideal para barras de navegação, alinhamento de botões e pequenos grupos de elementos.



style.css

```
.navegacao {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Eixo principal ↔

justify-content distribui no eixo principal.

align-items alinha no eixo transversal.

Grid: pense em linhas e colunas.

Ideal para galerias, listas de cards e áreas de um painel.

01 / HTML

Um card em cada coluna.

02 / CSS

Um card em cada coluna.

03 / LAYOUT

Um card em cada coluna.

style.css

```
.cursos {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
```

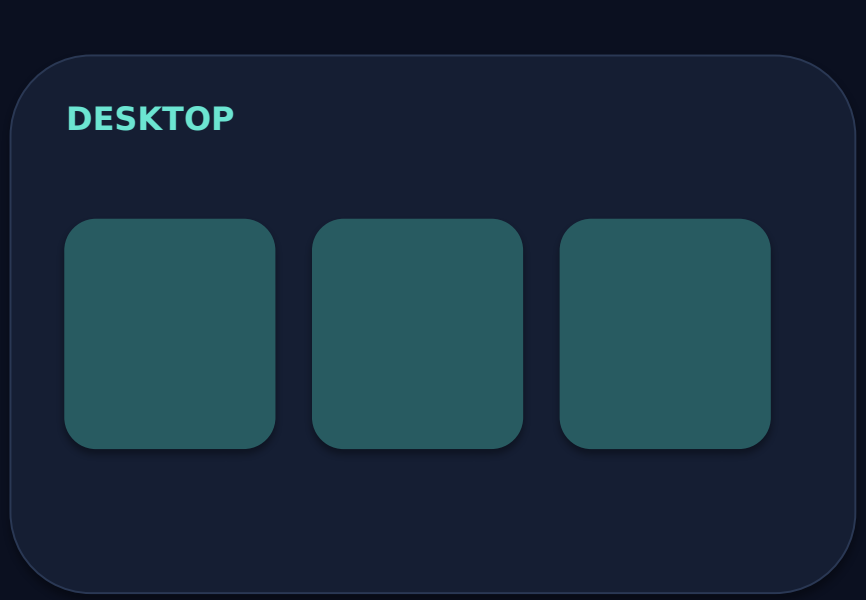
3 colunas proporcionais.

16px entre os cards.

Layout definido no contêiner.

Responsividade: adapte o layout ao espaço.

Não é apenas encolher: é reorganizar o conteúdo para manter leitura e uso confortáveis.





style.css

```
@media (max-width: 700px) {
  .cursos {
    grid-template-columns: 1fr;
  }
}
```

Viewport + dimensões flexíveis + media queries + testes em larguras diferentes.

Uma boa interface também precisa ser acessível.

01 **Semântica e hierarquia**

Títulos coerentes, regiões claras e HTML pelo significado.

02 **Rótulos e alternativas**

Associe label aos campos e use alt adequado em imagens.

03 **Leitura e contraste**

Texto legível, espaço confortável e informação além da cor.

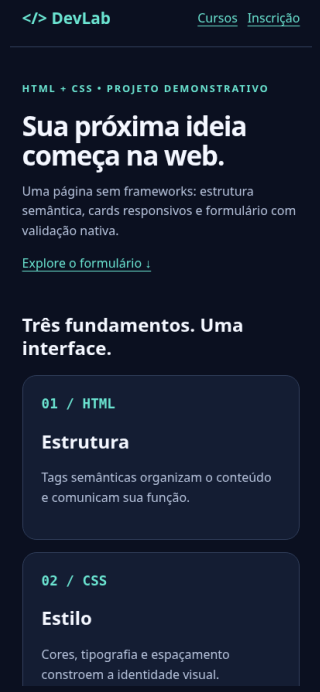
04 **Teclado e foco**

Mantenha a ordem de navegação e um indicador de foco visível.

Projeto DevLab: os conceitos juntos.

Página de cursos feita com HTML e CSS, sem frameworks ou dependências externas.





O que observar

- ✓ HTML semântico
- ✓ Navegação com Flexbox
- ✓ Cards com Grid
- ✓ Formulário com labels
- ✓ Layout responsivo

Capturas reais do exemplo.html em 1280px e 390px de largura.

Demonstração ao vivo: três verificações.

01 **Redimensionar a janela**

Acima de 700px: três cards. Até 700px: uma coluna.

02 **Navegar usando Tab**

Observe os links, os campos e o contorno de foco.

03 **Testar o formulário**

Envie vazio; teste um e-mail inválido; depois use dados fictícios válidos.

O essencial para levar com você.

HTML

Estruture pelo significado.

Tags semânticas, hierarquia, links e formulários.

CSS

Estilize com regras claras.

Seletores, classes, IDs, cascata e box model.

LAYOUT

Planeje para várias telas.

Flexbox, Grid, media queries e testes de uso.

DESAFIO À TURMA **Qual a diferença entre class e id? E entre margin e padding?**

Fontes para aprofundar e praticar.

Documentação técnica para conferir conceitos, exemplos e boas práticas.

MDN • HTML	developer.mozilla.org/pt-BR/docs/Web/HTML
MDN • CSS	developer.mozilla.org/pt-BR/docs/Web/CSS
MDN • Formulários	developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms
MDN • Layout responsivo	developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
W3C WAI • Acessibilidade	w3.org/WAI/tutorials/

PRÓXIMO PASSO / Edite uma cor, crie outro card e teste novamente em uma tela estreita.